

The VMS Algorithm

1. Problem

Solve the convection–diffusion problem with homogeneous Dirichlet data,

$$\mathcal{L}u = -\varepsilon \Delta u + \mathbf{b} \cdot \nabla u + cu = f, \quad u|_{\partial\Omega} = 0,$$

with the associated bilinear form $a(u, v) = \varepsilon(\nabla u, \nabla v) + (\mathbf{b} \cdot \nabla u, v) + c(u, v)$. The coarse (unknown) space is the maximal-continuity spline space $\bar{V} = S_{p-1}^p \cap H_0^1$ on the current mesh, a tensor product of two such spaces in two dimensions.

The unknown is split as $u = \bar{u} + u'$: a coarse part $\bar{u} \in \bar{V}$ and a fine-scale part u' living in a space V_h' that is orthogonal to $\bar{V}$. The fine scale is not modelled by a stabilisation parameter; it is computed, and the same computed object serves twice — once inside the solver, once as the error estimator.

2. The adaptive loop

Algorithm 1 — adaptive VMS loop

input: degree p , initial mesh, marking fraction θ , stopping tolerance

repeat for each refinement step:

1. Build the coarse space $\bar{V}$ and its operators on the current mesh.
2. Build the fine-scale space V_h' by $8 \times$ subdivision [Algorithm 2].
3. Factorise the fine-scale saddle-point matrix *once*; the result is the operator G' .
4. Solve the VMS coarse problem $\Rightarrow u_h$ [Algorithm 3].
5. Estimate: $v = G'(f - \mathcal{L}u_h)$, $\eta_K = \|v\|_{L^2(K)}$ [Algorithm 4].
6. Mark and refine [Algorithm 5]; in uniform mode, refine everything.

until the step budget, the degree-of-freedom budget or the tolerance on η is reached.

report: dof, $\|u - u_h\|_{L^2}$, $\eta = (\sum_K \eta_K^2)^{1/2}$, $I_{\text{eff}} = \eta/\|u - u_h\|$, observed order.

Steps 3–5 share one factorisation, so once the solve is done the estimator costs one extra solve.

3. Construction of the fine scale

The fine scale is built by dividing the coarse space 8 times: every coarse element is cut into 8 equal sub-cells, and the *same* spline degree and continuity are laid down on that sub-mesh. That refined space, made orthogonal to the coarse space, is V_h' .

Algorithm 2 — the fine-scale space V'_h

input: the coarse mesh, degree p , subdivision factor m ($m = 8$)

(a) **Sub-mesh.** For every coarse element $[x_e, x_{e+1}]$ insert $m - 1$ interior points, so that the element becomes m equal sub-cells. In two dimensions do this in x and in y separately, giving a tensor sub-mesh with m^2 sub-cells per coarse element.

(b) **Fan (layer resolution).** Add geometrically halved cells towards the outflow node, until the smallest cell is $\lesssim \varepsilon/|\mathbf{b}|$: in one dimension into every element's downstream node, in two dimensions only into the outflow boundary of the domain (grading every element starves the interior). This is what lets V'_h actually hold a layer.

(c) **Fine space.** $V_h = S_{p-1}^p(\text{sub-mesh}) \cap H_0^1$, with $p + 2$ Gauss points per sub-cell. By construction $\bar{V} \subset V_h$.

(d) **Orthogonality constraint** $Bv = 0$:

- 2D: $(\nabla v, \nabla \bar{v}) = 0 \quad \forall \bar{v} \in \bar{V}$; in matrix form $B = C^\top K_f$, with C the coarse basis written in the fine basis (prolongation) and K_f the sub-mesh stiffness matrix.

- 1D, $p \geq 2$: $(v, w) = 0 \quad \forall w \in S_{p-3}^{p-2}(\text{coarse mesh})$.

- 1D, $p = 1$: $v = 0$ at the interior coarse knots (the classical bubble).

Dependent rows of B are dropped by a rank-revealing QR.

$V'_h = \{v \in V_h : Bv = 0\}$

(e) **Sub-mesh operator, with SUPG added.** Assemble the operator on the sub-mesh,

$$A_f = \varepsilon K_f + \mathbf{b} \cdot C_f + c M_f, \quad A_f \leftarrow A_f + P^\top (\tau W) L,$$

the second term being SUPG *on the sub-mesh*: P is the streamline derivative $\mathbf{b} \cdot \nabla$ of the sub-mesh basis, L the full operator $\mathcal{L}$ applied to it, W the quadrature weights and τ the usual sub-cell SUPG parameter. The load is upwinded to match, $F(r) \leftarrow F(r) + P^\top (\tau W) r$.

(f) **One factorisation.** Impose $Bv = 0$ by Lagrange multipliers and factorise once:

$$\mathcal{K} = \begin{bmatrix} A_f & B^\top \\ B & 0 \end{bmatrix}.$$

output: the operator G' . Given any residual r , $G'r$ is *one back-substitution*

$$\begin{bmatrix} A_f & B^\top \\ B & 0 \end{bmatrix} \begin{bmatrix} v \\ \lambda \end{bmatrix} = \begin{bmatrix} F(r) \\ 0 \end{bmatrix}, \quad F(r)_j = \int r N_j,$$

and many right-hand sides can be pushed through at once as columns.

What the constraint does. It removes from v everything the coarse space can already represent, so V'_h carries only what $\bar{V}$ misses. Without it the two scales would overlap, the coarse solution would be corrected twice by the same content, and the fine-scale field would no longer measure the error. It is the constraint — not the size of the sub-mesh — that makes v a fine scale rather than just a finer solution.

Choice of the subdivision factor m . In one dimension $m = 8$ is the working value, and the effectivity is insensitive to it once the sublayer is captured. In two dimensions each coarse element carries m^2 sub-cells, so the same m is far more expensive and a smaller one is used:

- $m = 2$ **per direction** when the layer sits on the boundary of the domain. There the geometric fan of step (b), not the uniform subdivision, is what resolves the layer, so the uniform part only has to provide enough room inside a coarse element and two sub-cells per direction suffice.
- $m = 6$ **per direction at $p = 1$.** Linear splines are the poorest fine space: each sub-cell contributes a single hat function, and the orthogonality constraint removes most of them, so the subdivision has to be made finer to leave a usable V'_h behind.
- $m = \lceil h_{\min}/\sqrt{\varepsilon} \rceil$, **capped at about 12**, when the layer lies *inside* the domain. There is no

boundary to fan into, so the uniform subdivision must resolve the layer by itself, and m is chosen so that one sub-cell is about as wide as the layer: $h_{\min}$ is the smallest element of the current mesh and $\sqrt{\varepsilon}$ the layer half-width. On a coarse mesh this asks for many sub-cells; as refinement drives $h_{\min}$ towards $\sqrt{\varepsilon}$ it falls back to small values on its own. The cap keeps the m^2 cost bounded.

4. The solver

Algorithm 3 — the VMS coarse problem

Substituting $u' = G'(f - \mathcal{L}\bar{u})$ and testing with a coarse weight gives the coarse problem

$$a(\bar{u}, \bar{v}) + a(G'(f - \mathcal{L}\bar{u}), \bar{v}) = (f, \bar{v}) \quad \forall \bar{v} \in \bar{V}.$$

1. Assemble the coarse matrix A and load F as usual.
2. $g_f = a(G'f, \bar{v})$ — one application of G' to the source.
3. For each coarse basis function φ_j (in blocks of columns):
 - (i) evaluate $\mathcal{L}\varphi_j$ at the *sub-mesh* quadrature points;
 - (ii) $w_j = G'(-\mathcal{L}\varphi_j)$ — one back-substitution per column;
 - (iii) column j of the correction: $B_{ij} = a(w_j, \varphi_i)$.
4. Solve the small coarse system *directly*: $(A + B)\bar{u} = F - g_f$.

No stabilisation parameter enters here: the correction B is the computed fine-scale response, not a model of it. Solving step 4 iteratively instead stalls on strongly graded meshes at high p and silently corrupts u_h , which then makes the estimator look wrong when it is not.

5. The estimator

Algorithm 4 — the fine-scale error estimator

1. Strong residual of the coarse solution at the sub-mesh quadrature points: $r = f - \mathcal{L}u_h$.
2. $v = G'r$ — one back-substitution against the factorisation already built in Algorithm 2.
3. Gather the L^2 mass back onto the *coarse* elements: $\eta_K = (\sum_{q \in K} w_q v(x_q)^2)^{1/2}$.
4. $\eta = (\sum_K \eta_K^2)^{1/2}$, $I_{\text{eff}} = \eta / \|u - u_h\|_{L^2}$.

The solver and the estimator use the same V'_h and the same factorisation, so v is the fine-scale part of u itself: η_K is a local certificate for the error, not a heuristic indicator.

6. Marking and refinement

Algorithm 5 — mark and refine

Dörfler: sort η_K decreasing and take the smallest set with $\sum \eta_K^2 \geq \theta \sum_K \eta_K^2$; the alternative *maximum* strategy takes $\eta_K \geq \theta \max \eta_K$. Optionally cap the marked set at a fixed fraction of the mesh.

1D: bisect every marked element.

2D, directional marking on the tensor mesh: score knot line i in x by $(\sum_j \eta_{ij}^2)^{1/2}$ and likewise in y , run Dörfler on the two one-dimensional scores, and bisect the marked intervals in x and y independently. Marking elements instead refines a cross through the mesh and costs several times the degrees of freedom for the same error.
